

# Gen AI / LLM / Agentic AI Internship Assignment

*Build a Reasoning, Tool-Using Insight Chatbot with LangGraph*

## 1. Overview

---

We're hiring two interns to work on generative AI, LLM, and agentic AI systems. This assignment is designed to evaluate how you think about agent design, not just whether you can call an LLM API. It is scoped to be completable in a few focused days by someone with working Python and LLM-app experience.

You will build a chatbot, orchestrated with LangGraph, that behaves like a reasoning analyst: given a question, it decides what it needs to do, selects the right tool(s) to get there, and reports back an insight in plain language — not just a raw number or a wall of retrieved data.

## 2. Objective

---

- Design and implement a multi-step agent using LangGraph (state graph, nodes, conditional edges).
- Expose a visible reasoning process — the agent's intermediate thinking/plan should be inspectable, not hidden inside one opaque LLM call.
- Give the agent at least three distinct tools and have it choose between them correctly based on the user's query.
- Have the agent generate insights (trends, comparisons, anomalies, summaries) rather than just returning raw data.
- Wrap all of this in a chatbot interface and host it somewhere publicly reachable.

## 3. Assignment Scenario

---

Build “Insight Copilot” — a chatbot that helps a user explore and understand a dataset through natural-language conversation.

Pick ONE dataset from the shortlist in Section 4 (“Sample Datasets”) below, or propose an equivalent public dataset of your own — just tell us why.

The chatbot should let a user ask questions in natural language such as:

- *“What were the top 3 products by revenue last quarter?”*
- *“Is there a seasonal trend in sales for category X?”*
- *“Compare region A and region B performance and tell me what's driving the difference.”*
- *“Summarize anything unusual in this data.”*

The agent should reason about what's actually being asked, decide which tool(s) can answer it, execute them, and then synthesize a clear, human-readable insight — not just dump a table or a number.

## 4. Sample Datasets

---

Use one of the datasets below. They're ordered roughly easy → harder; pick based on how much of a stretch you want the assignment to be. All are free and require no special access.

| Dataset                                    | What it is                                                                                                                               | Difficulty  | Link                                                                                                                                                    |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Global Superstore Sales Dataset            | Multi-country variant of Superstore; adds a geography dimension for region-comparison questions.                                         | Easy        | <a href="https://kaggle.com/datasets/akelvarghese/global-superstore-sales-dataset">kaggle.com/datasets/akelvarghese/global-superstore-sales-dataset</a> |
| Superstore Sales (Time-Series Forecasting) | Same Superstore family, framed around date-indexed sales — nudges toward trend/seasonality questions.                                    | Easy–Medium | <a href="https://kaggle.com/datasets/rohitsahoo/sales-forecasting">kaggle.com/datasets/rohitsahoo/sales-forecasting</a>                                 |
| Brazilian E-Commerce Dataset by Olist      | ~100k real orders split across multiple related CSVs (orders, products, customers, reviews, sellers, geolocation). Genuinely relational. | Medium–Hard | <a href="https://kaggle.com/datasets/olistbr/brazilian-ecommerce">kaggle.com/datasets/olistbr/brazilian-ecommerce</a>                                   |

## 5. Technical Requirements

---

### 5.1 Orchestration — LangGraph

- Implement the agent as a LangGraph StateGraph (or equivalent current LangGraph API) — not a single linear LLM call, and not a bare LangChain AgentExecutor with no graph structure.
- Include at least one conditional edge (routing logic that depends on the state, e.g. “does this query need a tool at all?” or “which tool category does this fall into?”).
- Define and use a typed state object that's carried through the graph.

### 5.2 Reasoning

- Before acting, the agent should articulate a short plan/rationale (e.g. “This question needs sales data grouped by region, then a comparison — I'll query the data tool, then summarize.”).
- This reasoning trace should be visible somewhere in the UI (a collapsible “show reasoning” panel, a sidebar, or inline — your call), so an evaluator can see how the agent got to its answer.

### 5.3 Tool selection

Implement at least three tools, and the agent must genuinely choose between them based on the query (not always call all three). Suggested tools — pick any three or more:

- A data query tool (pandas / SQL over the dataset)
- A calculation tool (aggregations, growth rates, statistics)
- A charting/visualization tool (returns a chart or chart spec)
- A web search tool (for context outside the dataset, if relevant)
- A code-execution tool (for ad hoc analysis the other tools don't cover)

Bonus points for handling queries that need more than one tool in sequence, and for gracefully saying “I don't have a tool for that” rather than hallucinating an answer.

## 5.4 Insight generation

- Final responses should read like an analyst's answer: a direct answer to the question, the supporting numbers, and a one-line “why this matters” or “what stands out” takeaway.
- Avoid dumping raw tool output verbatim as the final answer — the LLM should synthesize it.

## 5.5 Chatbot interface

- Any chat UI is fine: Streamlit, Gradio, Chainlit, a simple React/Next.js frontend, etc.
- Must support multi-turn conversation (follow-up questions should have access to prior context).

## 6. Allowed Tools & Stack

---

You have full freedom here — use whatever tools, services, and models you're comfortable with. Some pointers, not requirements:

- LLM: any provider is fine — Anthropic, OpenAI, Google, or an open-source model via Groq/Together/Ollama. Use free tiers/trial credits where possible; we're not expecting you to spend money.
- Orchestration: LangGraph is required for the agent core. LangChain components underneath are fine.
- UI: Streamlit / Gradio / Chainlit are the fastest paths, but any framework works.
- Hosting: Streamlit Community Cloud, Hugging Face Spaces, Render, Railway, Vercel, or your own server — anything that gives us a public URL.
- Data: a bundled CSV/SQLite file in the repo is simplest — no need for a live database.

## 7. Deliverables

---

| Item                 | Details                                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Hosted chatbot URL   | A public link where we can chat with your agent directly. No local setup should be required to evaluate it.                            |
| GitHub repository    | Public repo (or private with access granted) containing all source code, the LangGraph definition, and a requirements/dependency file. |
| README               | Setup instructions, architecture summary, list of tools implemented, known limitations, and any assumptions you made.                  |
| Architecture diagram | A simple diagram (hand-drawn, Excalidraw, Mermaid, draw.io — anything) of your LangGraph state graph: nodes, edges, and routing logic. |
| Short write-up       | Half a page to one page: design decisions, trade-offs, what you'd improve with more time.                                              |

## 8. Hosting & Access

---

Your app must be reachable at a public URL for us to test without running any code locally. Practical notes:

- Free hosting is completely sufficient — Streamlit Community Cloud and Hugging Face Spaces are the easiest for a Streamlit/Gradio app and are free.
- Do not commit API keys to the repo. Use the hosting platform's secrets manager (e.g. Streamlit Cloud's “Secrets”, HF Spaces' “Repository secrets”).
- If the app needs to “wake up” from a free-tier sleep, mention that in the README so we don't think it's broken.
- If your GitHub repo is private, add the reviewer(s) we tell you about as collaborators.

## 9. Evaluation Rubric

---

| Criterion              | What we're looking for                                                                                                                     | Weight |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|--------|
| LangGraph architecture | Clean state graph: sensible nodes/edges, conditional routing, persisted/typed state (not a single giant prompt pretending to be an agent)  | 25%    |
| Reasoning quality      | Visible, coherent reasoning trace before tool calls or final answers; agent explains why it picked a path, not just what it did            | 20%    |
| Tool selection         | Agent reliably picks the right tool for the query and gracefully skips tools when none are needed; handles ambiguous or multi-step queries | 20%    |
| Insight generation     | Outputs are not just raw numbers/data dumps — the agent synthesizes trends, comparisons, anomalies and states them in plain language       | 15%    |
| Engineering quality    | Code readability, error handling, secrets management, repo structure, README quality                                                       | 10%    |
| Hosting & UX           | App is actually reachable, doesn't crash, has a usable chat UI                                                                             | 10%    |

Note: we care more about how you reasoned about the agent design than about visual polish. A slightly rough UI with a genuinely well-structured LangGraph agent will score higher than a beautiful UI wrapping a single hidden prompt.

## 10. Timeline

---

**Submission deadline: within 5 calendar days of receiving this assignment**

## 11. Submission Instructions

---

1. Push your code to a GitHub repository (public, or private with access granted).
2. Deploy the app and confirm the public URL works in an incognito/private browser window.
3. Reply to this assignment email/message with: (a) hosted URL, (b) repo link, (c) your short write-up.

## 12. Questions / Ambiguity

---

Some parts of this brief are intentionally open (which dataset, which tools, which UI framework). That's deliberate — part of what we're evaluating is how you scope and make decisions with limited direction. If something is genuinely blocking, reach out.

---

*Good luck — we're looking forward to seeing how you think, not just what you ship.*